AI Personal Style & Wellness Advisor

Complete Full Stack + ML + File Structure + Development Master Guide

1. Project Idea

```
AI Personal Style & Wellness Advisor is a full stack AI-powered web application
that analyzes face shape, facial features, color tones, preferences, profession,
and style interests to provide personalized recommendations for:
- Hairstyles
- Outfits
- Glasses
- Accessories
- Color palettes
- Grooming suggestions

The project combines:
- Computer Vision
- Machine Learning
- Recommendation Systems
- Full Stack Development
```

2. Tech Stack

```
Frontend:
- React.js
- Tailwind CSS
- Axios
- React Router DOM

Backend:
- FastAPI
- SQLAlchemy
- JWT Authentication

Database:
- SQLite
- PostgreSQL (future)

Machine Learning:
- TensorFlow
- PyTorch
- OpenCV
- MediaPipe

Deployment:
- Vercel
- Render
```

3. Frontend File Structure

```
frontend/
■
■■■ public/
■   ■■■ logo.png
■   ■■■ favicon.ico
■
■■■ src/
■   ■■■ components/
■   ■   ■■■ Navbar.jsx
■   ■   ■■■ Footer.jsx
■   ■   ■■■ Sidebar.jsx
■   ■   ■■■ Loader.jsx
■   ■   ■■■ ImageUploader.jsx
■   ■   ■■■ OutfitCard.jsx
■   ■   ■■■ HairstyleCard.jsx
```

```
■   ■   ■■■ ProtectedRoute.jsx
■   ■
■   ■■■ pages/
■   ■   ■■■ Home.jsx
■   ■   ■■■ Login.jsx
■   ■   ■■■ Register.jsx
■   ■   ■■■ Dashboard.jsx
■   ■   ■■■ UploadPhoto.jsx
■   ■   ■■■ FaceAnalysis.jsx
■   ■   ■■■ OutfitRecommendations.jsx
■   ■   ■■■ HairstyleRecommendations.jsx
■   ■   ■■■ ColorPalette.jsx
■   ■   ■■■ History.jsx
■   ■   ■■■ Profile.jsx
■   ■
■   ■■■ services/
■   ■   ■■■ api.js
■   ■   ■■■ authService.js
■   ■   ■■■ analysisService.js
■   ■   ■■■ recommendationService.js
■   ■
■   ■■■ App.js
■   ■■■ main.jsx
■   ■■■ index.css
```

4. Backend File Structure

```
backend/
■
■■■ app/
■   ■■■ routers/
■   ■   ■■■ auth.py
■   ■   ■■■ users.py
■   ■   ■■■ analysis.py
■   ■   ■■■ recommendations.py
■   ■   ■■■ admin.py
■   ■
■   ■■■ models/
■   ■   ■■■ user_model.py
■   ■   ■■■ analysis_model.py
■   ■   ■■■ recommendation_model.py
■   ■   ■■■ feedback_model.py
■   ■
■   ■■■ schemas/
■   ■   ■■■ user_schema.py
■   ■   ■■■ auth_schema.py
■   ■   ■■■ recommendation_schema.py
■   ■
■   ■■■ services/
■   ■   ■■■ auth_service.py
■   ■   ■■■ face_service.py
■   ■   ■■■ recommendation_service.py
■   ■   ■■■ history_service.py
■   ■
■   ■■■ utils/
■   ■   ■■■ jwt_handler.py
■   ■   ■■■ image_utils.py
■   ■   ■■■ file_handler.py
■   ■
■   ■■■ database.py
■   ■■■ config.py
■   ■■■ main.py
```

5. ML Model Structure

```
ml_models/
■
■■■ datasets/
■■■ trained_models/
■■■ face_detection.py
■■■ landmark_detection.py
■■■ face_shape_classifier.py
■■■ recommendation_engine.py
■■■ outfit_matcher.py
■■■ hairstyle_matcher.py
```

```
■■■ color_analysis.py
```

6. Complete User Flow

```
User Opens Application
↓
Registers/Login
↓
Uploads Face Image
↓
Frontend Sends Image to Backend
↓
FastAPI Receives Image
↓
ML Model Detects Face
↓
Landmarks Extracted
↓
Face Shape Predicted
↓
Recommendation Engine Generates Suggestions
↓
Results Stored in Database
↓
Frontend Displays Personalized Recommendations
```

7. Database Tables

```
Main Tables:
- users
- face_analysis
- recommendations
- user_preferences
- feedback
- recommendation_history
- admin_logs
```

8. API Design

```
POST /api/auth/register
POST /api/auth/login
GET /api/users/profile
POST /api/analyze-face
GET /api/recommend/hairstyle
GET /api/recommend/outfits
GET /api/recommend/colors
GET /api/history
POST /api/feedback
GET /api/admin/users
```

9. Authentication Flow

```
User Registers
↓
Password Hashed
↓
Saved in Database
↓
User Logs In
↓
JWT Token Generated
↓
Frontend Stores Token
↓
Protected Routes Accessible
```

10. How React Connects with FastAPI

```
Install Axios:
npm install axios

api.js:

import axios from 'axios';

const API = axios.create({
   baseURL: 'http://localhost:8000/api'
});

export default API;
```

11. How ML Model Connects

```
Frontend Uploads Image
↓
Backend Receives Image
↓
analysis.py calls face_shape_classifier.py
↓
Model predicts face shape
↓
recommendation_engine.py generates recommendations
↓
Backend sends JSON response
```

12. How to Run Frontend

```
cd frontend
npm install
npm run dev

Frontend URL:
http://localhost:5173
```

13. How to Run Backend

```
cd backend
uvicorn app.main:app --reload

Backend URL:
http://localhost:8000

Swagger Docs:
http://localhost:8000/docs
```

14. Development Roadmap

```
PHASE 1:
Setup folders and project structure

PHASE 2:
Create frontend pages

PHASE 3:
Create backend APIs

PHASE 4:
Build face detection system

PHASE 5:
Build face shape prediction
PHASE 6:
```

```
Build recommendation engine

PHASE 7:
Connect frontend + backend + ML

PHASE 8:
Add authentication

PHASE 9:
Add database

PHASE 10:
Deploy online
```

15. Final Project Outcome

```
Your final project becomes:

- Full Stack AI Application
- Computer Vision System
- Personalized Fashion Assistant
- Recommendation Engine
- Portfolio-Level ML Project
- Startup MVP Prototype
- Industry-Level Architecture Project
```